

Compiler Homework 2: Parser

by: B123245006陳展甦

System info:

Lex version: flex 2.6.4

Yacc version: bison (GNU Bison) 2.3

Operating Platform: macOS 26.5

Executing Method:

Install:

- macOS: brew install flex
- Linux: sudo apt-get install flex bison

Compile:

- macOS: make mac
- Linux: make

Run:

- ./parser < TEST_FILE.java

Handle Issues in this specification:

1. New .l file was modified from the previous homework to work with the new .y file.
2. The scope and symbol table management was implemented to handle variable declarations and references, including nested scopes and unused variable warnings. This involved creating a stack of Scope structs to represent the current scope and its parent scopes, and a linked list of Symbol structs to represent the identifiers declared in each scope. The parser was modified to call the appropriate functions for managing the symbol table when entering and leaving scopes, declaring identifiers, and marking identifiers as used when they are referenced in the code.
3. Classes are implemented with fields and methods. The parser can handle class declarations, including the syntax for declaring fields and methods, and manage the symbol table for class members. The fields are treated as variables declared in the class scope. The parser also handles the syntax for method declarations, including the parameter list and method body, and manages the symbol table for method parameters and local variables.
4. Field declarations include variable declarations with optional initializers. The parser can handle the syntax for field declarations, including the optional initializer, and manage the symbol table for class fields. When a field is declared, it is added to the symbol table for the class scope, and if it has an initializer, the parser processes the initializer expression to mark any identifiers used in it as referenced.
5. Method declarations include parameters and a method body. The body can contain variable declarations and statements. The parser can handle the syntax for method declarations, including the parameter list and method body, and manage the symbol table for method parameters and local variables. The parser also handles the syntax for statements within the method body, such as variable declarations, assignments, and control flow statements.

6. Statements include variable declarations, assignments, and control flow statements. The parser can handle the syntax for these statements and manage the symbol table accordingly. Variable declarations are processed to add identifiers to the symbol table, while assignments and control flow statements are processed to mark identifiers as used when they are referenced in the code.
7. Expressions include literals, identifiers, and method invocations. The parser can handle the syntax for these expressions and manage the symbol table accordingly. Literals are processed as constant values, while identifiers are processed to mark them as used when they are referenced in expressions. Method invocations are parsed through the name and argument list rules, and any identifiers used inside them are marked as referenced.
8. Errors are reported for redeclarations of identifiers in the same scope, unused variable warnings, and general syntax errors. The parser checks for these errors when processing declarations and references, and prints appropriate error messages when they are detected. Redeclarations are reported when an identifier is declared more than once in the same scope, while unused variable warnings are reported for variables that are declared but never referenced in the code.
9. AI and internet is used to help understanding the specification and providing suggestions for implementation

Problems encountered while writing this assignment:

1. Handling error was tricky, especially for syntax errors and redeclarations. I had to carefully check the conditions ensure proper error reporting.
2. yyerror and yyerrok were used to handle syntax errors in if and for statements. I had to make sure to call yyerrok after reporting the error to prevent cascading errors.
3. yacc shows shift/reduce conflicts for multiple statements. I had to use precedence rules to resolve these conflicts and ensure the correct parsing of statements.

Results of executing all of test files:

```

~/g/SchoolC/c/HW2 main 130 ?? ./parser < test1.jav
a
line 1: /* Test file: Perfect test file
line 2: * Compute sum = 1 + 2 + ... + n
line 3: */
line 4: class sigma {
line 5: // "Final" should have const_expr
line 6: final int n = 10;
line 7: int sum, index;
line 8:
line 9: main()
line 10: {
line 11:     index = 0;
line 12:     sum = 0;
line 13:     while (index <= n)
line 14:     {
line 15:         sum = sum + index;
line 16:         index = index + 1;
line 17:     }
line 18:     print(sum);
line 19: }
line 20: }

~/g/SchoolC/c/HW2 main 130 ?? ./parser < test2.jav
a
line 1: /*Test file: Duplicate declare variable in the
same scope*/
line 2: class Point
line 3: {
line 4:     static int counter ;
line 5:     int x, y ;
line 6:     /*Duplicate declare x*/
*****x' in the next line is a duplicated identifier i
n the current scope.*****
line 7:     int x ;
line 8:     void clear()
line 9:     {
line 10:         x = 0 ;
line 11:         y = 0 ;
line 12:     }
line 13: }

~/g/SchoolC/c/HW2 main 130 ??
~/g/SchoolC/c/HW2 main 130 ?? ./parser < test3.jav
a
line 1: /*Test file of Syntax error: Out of symbol. Bu
t it can go through*/
line 2: class Point {
line 3:     int z;
line 4:     char: 11, syntax error at "y"
line 5:     int x y;
line 6:     /*Need ', ' before y*/
line 7:     float w;
line 8: }
line 9: class Test {
line 10:     int d;
line 11:     Point p = new Point()
line 12:     /*Need ' ' at EOL*/
line 13:     int w,q;
line 14: }

~/g/SchoolC/c/HW2 main 130 ?? ./parser < test4.jav
a
line 1: /*Test file: Duplicate declaration in different
scope and same scope*/
line 2: class Point
line 3: {
line 4:     int x, y ;
line 5:     int p;
line 6:     boolean test()
line 7:     {
line 8:         /*Another x, but in different s
cope*/
line 9:     }
line 10:     int x;
line 11:     /*Another x in the same scope*/
*****x' in the next line is a duplicated identifier i
n the current scope.*****
line 12:     char x;
line 13:     {
line 14:         boolean w;
line 15:         /*Another w in the same scope*/
line 16:         int w;
line 17:         /*w' at line 13 is never used.*****
line 18:         }
line 19:     }
line 20:     class Test
line 21:     {
line 22:         /*Another p, but in different scopes*/
line 23:         Point p = new Point();
line 24:     }
line 25: }

~/g/SchoolC/c/HW2 main 130 ?? ./parser < test5.jav
a
line 1: class test5{
line 2:     int add(int a1, int a2){
line 3:         return (a1 + a2);
line 4:     }
line 5:     void main() {
line 6:         int x, y, z;
line 7:         for(int i=0;i<2;i++){
line 8:             if(i==0){
line 9:                 *****Else Without If at line 10, char 17*****
line 10:                 else
line 11:                 {
line 12:                     i = 1;
line 13:                 }
line 14:                 for(x = 0; x<5;x++){
line 15:                     y++;
line 16:                     x = add(x,y);
line 17:                     x = z(x,y);
line 18:                 }
line 19:             }
line 20:             print("x:"*x*y:**y);
line 21:             z = ( x + y ) * 5 / 2-- -y;
line 22:         }
line 23:     }
line 24: }
line 25: /* this is a comment // line// with some /* */
and
line 26: // delimiters */

~/g/SchoolC/c/HW2 main 130 ??
~/g/SchoolC/c/HW2 main 130 ?? ./parser < test6.jav
a
line 1: class test6{
line 2:     void sum(){
line 3:         //-----NEVER USED
line 4:         int sumxyz = x + y + z + S + x;
line 5:         *****sumxyz' at line 4 is never used.*****
line 6:     }
line 7:     void main() {
line 8:         int [] i = new int [1];
line 9:         for(i[0] = 0; i[0]<5; i[0]++)
line 10:             i[0]++;
line 11:     }
line 12: }
line 13: //-----NEW CLASS
line 14:     Point lowerLeft = new Point() ;
line 15: }
line 16: //-----ERROR CONDITION
line 17: char: 15, syntax error at "*"
line 18:     while(*/*/*/*){
line 19:         print("error!!");
line 20:     }
line 21: //-----CLASS DECLARE
line 22:     class Point {
line 23:         int x, y, z;
line 24:         *****LowerLeft' at line 14 is never used.*****
line 25:     }
line 26: }

```